

Final Project Report

Optimization of Graph Neural Networks under Graph Perturbations

Adam vs AdamW vs RMSProp vs AdaGrad vs SGD
on the Cora Citation Network

<u>Institution</u>	Faculty of Sciences Ben M'Sick
<u>University</u>	Hassan II University of Casablanca
<u>Department</u>	Mathematics and Computer Science
<u>Realized by</u>	Ossama Ashad Mohamed Amine Kar-any Mouhcine Ayar Hamza Elhaddaji Iliass Ouchida
<u>Supervisor</u>	Benabbou Fouzia
<u>Academic Year</u>	2025 – 2026
<u>Submission Date</u>	March 24, 2026

Abstract

This study investigates the impact of optimization algorithms on the training stability, convergence behavior, and adversarial robustness of Graph Convolutional Networks (GCNs). A two-layer GCN is trained on the Cora citation network and evaluated under a fixed experimental protocol across 13 conditions: one clean graph and 12 perturbed conditions spanning three perturbation types (edge removal, fake edge injection, and feature noise) at four severity levels (5%, 10%, 20%, 30%). Five optimizers are compared: Adam, AdamW, RMSProp, AdaGrad, and SGD. Results show that Adam achieves the best aggregate robustness with a mean accuracy of 0.702 across all conditions, while AdamW converges fastest, reaching within 1% of its best validation loss by epoch 157, at the cost of mild overfitting. RMSProp reaches the highest final validation accuracy (0.794) but is the slowest to converge. AdaGrad reaches its plateau very late (epoch 184) and exhibits the smallest train/validation gap (0.244), indicative of underfitting rather than overfitting. SGD fails to learn under the fixed learning rate protocol. Edge removal is the perturbation type best tolerated overall, though AdamW briefly leads Adam at the lowest severity (5%) before Adam takes over from 10% onward. Feature noise emerges as the universally most challenging perturbation type, collapsing all adaptive optimizers toward a similar degraded accuracy range regardless of their clean-graph performance. These findings suggest Adam as the most reliable optimizer for GCN training on citation networks under graph perturbations.



GitHub: <https://github.com/Maaskk/gnn-optimization-under-graph-perturbations-cora>

Dashboard: <https://maaskk.github.io/gnn-optimization-under-graph-perturbations-cora/?v=b923f04#top>

Table of Content

Abstract	2
1. Introduction	4
1.1 Project Objective	4
1.2 Optimizers Overview	4
1.3 Architecture: Graph Convolutional Network (GCN)	4
2. Literature Review	5
2.1 Graph Neural Networks	5
2.2 Optimization Algorithms	5
2.3 GNN Robustness	5
3. Methodology	6
3.1 Dataset: Cora Citation Network	6
3.2 GCN Architecture	6
3.3 Fixed Comparison Protocol	6
3.4 Perturbation Regimes	7
4. Results	8
4.1 Clean Graph Performance	8
4.2 Clean Accuracy & F1 Comparison	8
4.3 Loss Convergence and Generalization Gap	9
4.4 Convergence Speed	9
4.5 Validation Accuracy During Training	10
4.6 Gradient Stability	11
4.7 Feature Noise Robustness — Detailed Severity Analysis	11
4.8 Edge Removal Robustness — Detailed Severity Analysis	12
4.9 Fake Edge Addition Robustness — Detailed Severity Analysis	13
4.10 Training Time Analysis	14
4.11 Accuracy Drop Across Perturbation Severities	15
4.12 Aggregate Performance Across All Conditions	15
5. Discussion	17
5.1 Interpretation of Results	17
5.2 Strengths and Weaknesses per Optimizer	17
6. Limitations and Validity	19
7. Conclusion	20
7.1 Synthesis	Erreur ! Signet non défini.
7.2 Recommendations	Erreur ! Signet non défini.
8. Bibliography	21
9. Annexes	22

1. Introduction

1.1 Project Objective

Graph Neural Networks (GNNs) are deep learning architectures designed for graph-structured data, where nodes represent entities and edges encode relational information. Unlike images or text, graphs exhibit irregular topologies that require specialized learning mechanisms capable of aggregating neighborhood information.

This project investigates how optimization algorithms influence training stability, convergence speed, and robustness of GNNs when the input graph is subjected to perturbations. We train a two-layer Graph Convolutional Network (GCN) on the Cora citation network and compare five optimizers: Adam, AdamW, RMSProp, AdaGrad, and SGD.

The central research question is: which optimizer best preserves classification performance when graph structure or node features are corrupted by noise, and how does each optimizer's convergence behavior, gradient stability, and resource usage relate to its robustness?

1.2 Optimizers Overview

- Adam: combines momentum and adaptive per-parameter learning rates via first and second gradient moments.
- AdamW: extends Adam with decoupled weight decay applied directly to weights, improving generalization.
- RMSProp: adapts learning rates using a moving average of squared gradients, effective for non-stationary landscapes.
- AdaGrad: accumulates all past squared gradients, giving larger updates to infrequent features.
- SGD: classic first-order method with no adaptivity, requiring careful learning rate tuning.

1.3 Architecture: Graph Convolutional Network (GCN)

The GCN model (Kipf & Welling, 2017) propagates node information via spectral graph convolutions. Each layer aggregates features from direct neighbors, weighted by the normalized adjacency matrix. Our architecture uses two GCN layers ($1433 \rightarrow 16 \rightarrow 7$) with ReLU activation, 0.5 dropout during training, and LogSoftmax output for 7-class node classification.

2. Literature Review

2.1 Graph Neural Networks

Kipf & Welling (2017) [1] introduced Graph Convolutional Networks, approximating spectral graph convolutions with a first-order, localized propagation rule. Their two-layer GCN achieved 81.5% accuracy on Cora using only 140 labeled nodes, establishing it as the standard baseline for citation network classification and the architecture adopted in this study.

Hamilton et al. (2017) [2] proposed GraphSAGE, an inductive framework that samples and aggregates features from a fixed-size neighborhood rather than the full adjacency matrix, allowing it to generalize to unseen nodes on large, evolving graphs.

Velickovic et al. (2018) [3] introduced Graph Attention Networks (GAT), replacing GCN's fixed structural weighting with learned self-attention over neighbors — a property potentially useful for downweighting unreliable or spurious edges under noisy connectivity.

2.2 Optimization Algorithms

Kingma & Ba (2015) [4] proposed Adam, combining momentum with per-parameter adaptive learning rates from first and second gradient moment estimates, making it robust to sparse and noisy gradients and the default optimizer across most deep learning applications.

Loshchilov & Hutter (2019) [5] showed that Adam's standard L2 penalty does not behave like true weight decay, since it is rescaled by Adam's adaptive denominator. Their AdamW variant decouples weight decay from the gradient update, improving generalization in prior benchmarks.

Hinton et al. (2012) [6] introduced RMSProp to fix AdaGrad's vanishing learning rate, replacing its cumulative gradient sum with an exponentially decaying moving average — better suited to the noisy, non-stationary gradients typical of GCN training.

2.3 GNN Robustness

Zugner et al. (2018) [7] demonstrated that GNNs are highly vulnerable to small adversarial perturbations of structure or features, motivating this study's perturbation-based protocol, though the perturbations applied here are randomly sampled rather than adversarially optimized.

Fey & Lenssen (2019) [8] introduced PyTorch Geometric, the library used for all model implementations, dataset loading, and perturbation pipelines in this study, ensuring comparability with the broader GNN literature.

Together, this work validates the GCN architecture, identifies Adam, AdamW, and RMSProp as leading adaptive optimizers, and confirms GNN robustness as an open question — but none directly compares optimizer choice under a single fixed protocol. Sections 4 and 5 address this gap empirically.

3. Methodology

3.1 Dataset: Cora Citation Network

The Cora dataset is a standard benchmark for node classification on citation graphs, loaded via PyTorch Geometric (Planetoid):

Property	Value	Note
Nodes	2,708	Citation graph
Directed edges	10,556	—
Node features	1,433	Bag-of-words
Classes	7	Topic categories
Training nodes	140	5.2% of graph
Validation nodes	500	18.5%
Test nodes	1,000	36.9%

Table 1 — Cora Citation Network Properties

The small training set (140 nodes, 5.2% of the graph) makes optimization especially sensitive to hyperparameter choices and perturbations.

3.2 GCN Architecture

- Layer 1: GCNConv(1433 $\rightarrow$ 16) + ReLU + Dropout(0.5)
- Layer 2: GCNConv(16 $\rightarrow$ 7) + LogSoftmax
- Loss: Negative Log-Likelihood (NLLLoss) on 140 training nodes
- Metrics: Test accuracy, Macro F1-score, gradient L2 norm, and peak memory usage

3.3 Fixed Comparison Protocol

All experiments use an identical fixed protocol to isolate optimizer behavior as the only variable:

Parameter	Value	Detail
Model	2-layer GCN	PyTorch Geometric
Hidden channels	16	—
Dropout	0.5	Training only
Learning rate	0.01	Fixed, no scheduler
Weight decay	0.0005	L2 regularization
Epochs	200	Fixed budget
Seed	42	All experiments
Perturbation severities	5%, 10%, 20%, 30%	Per type

Table 2 — Experimental Protocol (identical across all 5 optimizers)

3.4 Perturbation Regimes

Three independent perturbation types are applied at four severity levels (5%, 10%, 20%, 30%):

1. Feature Noise: Gaussian noise $N(0, \text{severity})$ added to all 1,433 node feature dimensions.
2. Edge Removal: a random fraction of citation edges deleted from the graph.
3. Fake Edge Addition: spurious edges injected between random node pairs.

Each optimizer is evaluated on 13 conditions (1 clean + 12 perturbed), yielding 65 total experiment runs. A supplemental instrumented run additionally tracks per-epoch gradient L2 norm and peak memory usage on the clean graph for each optimizer.

4. Results

4.1 Clean Graph Performance

Table 3 summarizes all metrics on the unperturbed Cora graph:

Optimizer	Test Acc.	Macro F1	Final Loss	Train Time	Mean Acc.*	Rank
Adam	0.799	0.790	0.316	12.1s	0.702	1
RMSProp	0.795	0.787	0.381	41.7s	0.698	2
AdamW	0.799	0.790	0.058	21.1s	0.689	3
AdaGrad	0.719	0.719	1.411	24.5s	0.639	4
SGD	0.148	0.056	1.944	19.4s	0.172	5

Table 3 —Results on Clean Cora Graph (* Mean accuracy across all 13 tested conditions: 1 clean + 12 perturbed)

Adam and AdamW both achieve 0.799 test accuracy on the clean graph, the strongest result among all five optimizers. RMSProp follows closely at 0.795. AdaGrad reaches 0.719, and SGD trails the group substantially at 0.148 under the fixed learning rate of 0.01. This clean accuracy is consistent with the original GCN paper's reported 81.5%, the small difference reflecting our fixed single-protocol comparison setup rather than per-optimizer tuning.

4.2 Clean Accuracy & F1 Comparison

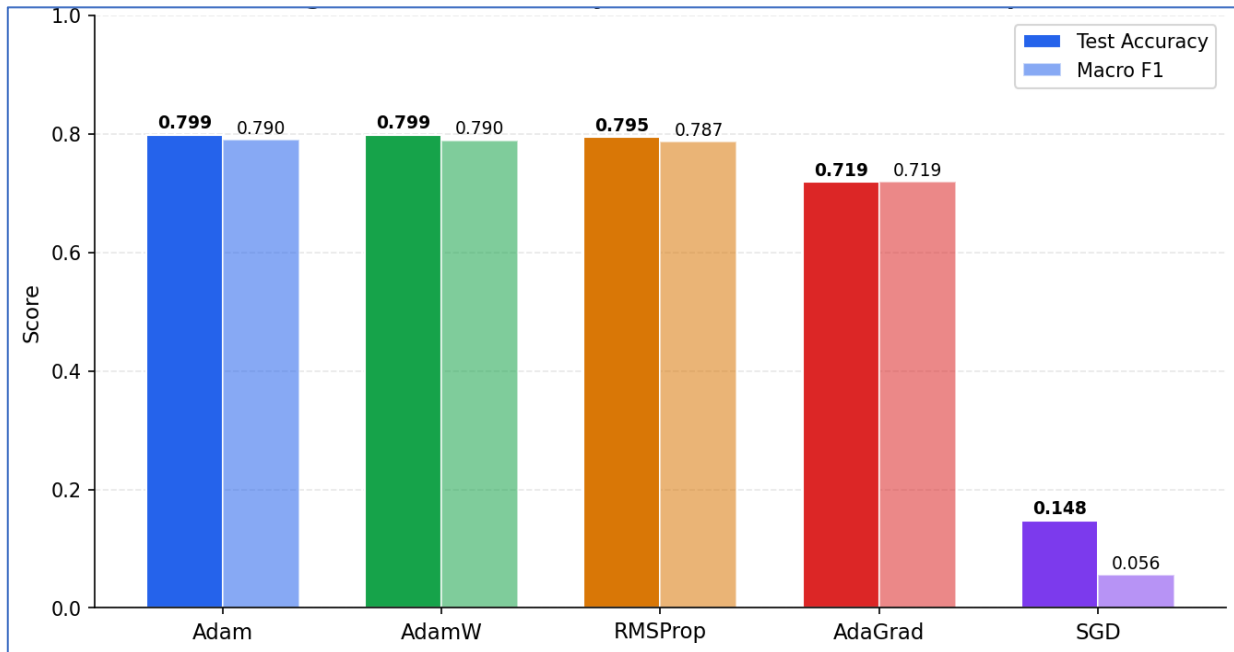


Figure 1 — Test Accuracy and Macro F1-score on Clean Cora Graph

Adam, AdamW, and RMSProp cluster closely in the 0.795–0.799 accuracy range. AdaGrad sits at 0.719. SGD remains the clear outlier at the bottom of the ranking across both metrics.

4.3 Loss Convergence and Generalization Gap

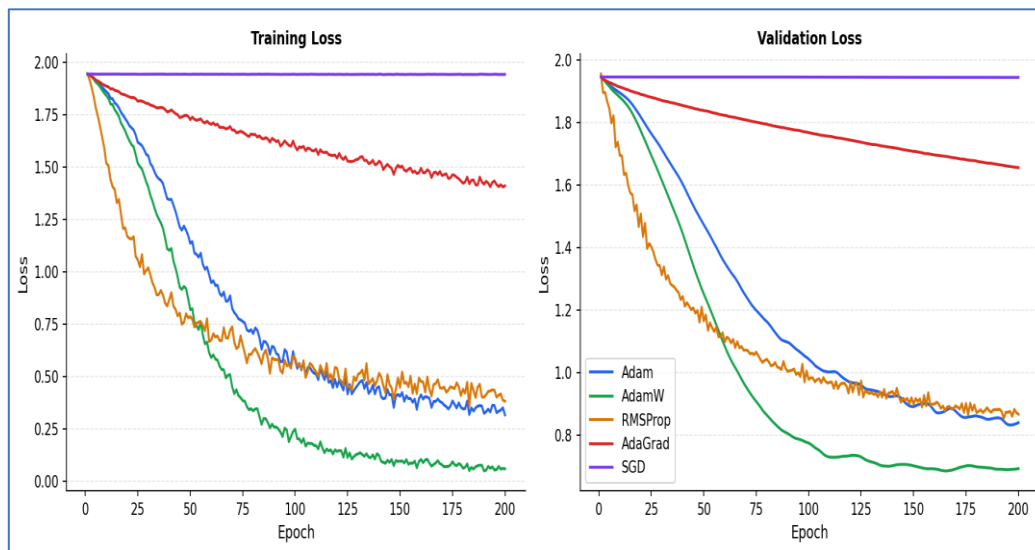


Figure 2 — Training and Validation Loss over 200 Epochs (Clean Graph)

Optimizer	Train Loss	Val Loss	Train/Val Gap	Final Val Acc.	Final Val F1
Adam	0.316	0.841	0.525	0.776	0.768
AdamW	0.058	0.693	0.636	0.782	0.769
RMSProp	0.381	0.867	0.486	0.786	0.775
AdaGrad	1.411	1.655	0.244	0.726	0.724
SGD	1.944	1.944	0.000	0.158	0.066

Table 4 — Final Train/Validation Metrics and Generalization Gap (Epoch 200)

The generalization gap reveals a different ranking than raw accuracy. AdamW shows the widest train/validation loss gap (0.636), despite having the lowest training loss of all optimizers (0.058) — a signature of mild overfitting to the 140-node training set. AdaGrad shows the smallest gap (0.244) among the optimizers that actually learn, suggesting it underfits rather than overfits. SGD's near-zero gap (-0.0003) is not a sign of good generalization but simply reflects that it never escapes its near-random initialization on either split.

4.4 Convergence Speed

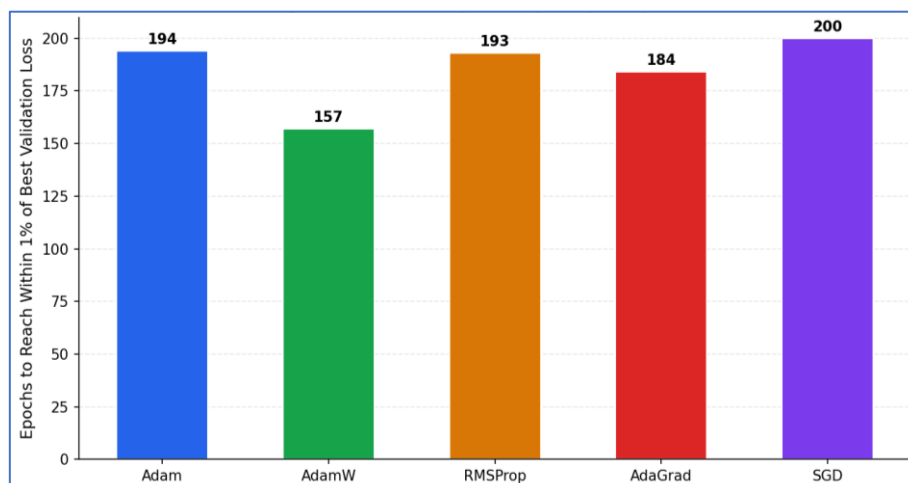


Figure 3 — Convergence Speed: Epochs to Reach Within 1% of Best Validation Loss

Optimizer	Best Val Loss Epoch	Epoch to 1% of Best	Best Val Accuracy	Status
Adam	196	194	0.782	Plateau reached
AdamW	165	157	0.792	Plateau reached
RMSProp	193	193	0.794	Plateau reached
AdaGrad	200	184	0.730	Plateau reached
SGD	200	200	0.166	No useful convergence

Table 5 — Convergence Status and Speed per Optimizer

AdamW converges fastest, reaching within 1% of its best validation loss by epoch 157 and its overall best validation loss at epoch 165. Adam takes considerably longer (epoch 194 to within 1%, epoch 196 for the actual best), despite matching AdamW's final accuracy. RMSProp converges slowly but steadily, reaching its best validation loss at epoch 193. AdaGrad reaches its plateau very late — at epoch 184 — still improving marginally until near the end of the 200-epoch budget. SGD is formally classified as "no useful convergence" — its validation loss at epoch 200 is statistically indistinguishable from its value at epoch 1.

4.5 Validation Accuracy During Training

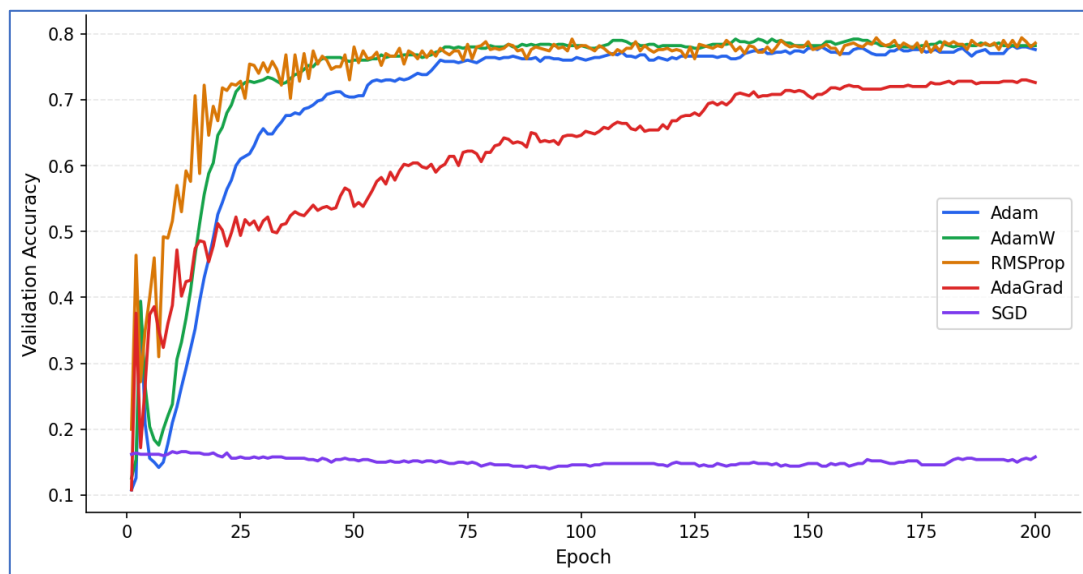


Figure 4 — Validation Accuracy Curves During Training (Clean Graph)

Adam and AdamW reach near-peak validation accuracy by epoch 50–60 and remain stable thereafter. RMSProp follows a similar trajectory with more oscillation but reaches the highest final validation accuracy (0.794) among all five optimizers. AdaGrad stabilizes at a lower accuracy plateau around epoch 100. SGD remains close to its starting accuracy throughout the entire training run.

4.6 Gradient Stability

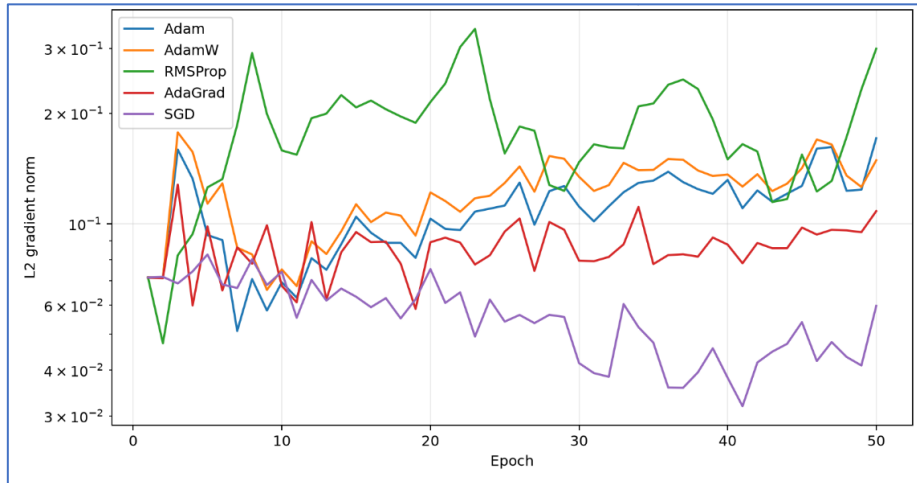


Figure 5 — Gradient L2 Norm During Training (Clean Graph, 5-epoch Rolling Mean)

Optimizer	Peak CPU RSS (MB)	Final Gradient L2 Norm	Final Training Loss
Adam	350.9	0.171	1.136
AdamW	350.9	0.149	0.819
RMSProp	350.9	0.300	0.786
AdaGrad	350.9	0.108	1.727
SGD	350.9	0.060	1.944

Table 6 — Peak Memory Usage and Final Gradient L2 Norm per Optimizer

RMSProp shows the highest final gradient L2 norm (0.300), consistent with its per-parameter normalization keeping gradient magnitudes from shrinking even late in training. AdaGrad shows the lowest gradient norm among the optimizers that learn successfully (0.108), reflecting its aggressively decaying effective learning rate. SGD's gradient norm (0.060) is misleadingly low — it reflects a model stuck near initialization rather than genuine convergence. Peak CPU memory usage is identical across all optimizers (350.9 MB), confirming that memory footprint in this setup is dominated by the fixed model size and Cora's feature matrix rather than by optimizer-specific state, despite Adam-family optimizers maintaining additional first and second moment buffers.

4.7 Feature Noise Robustness — Detailed Severity Analysis

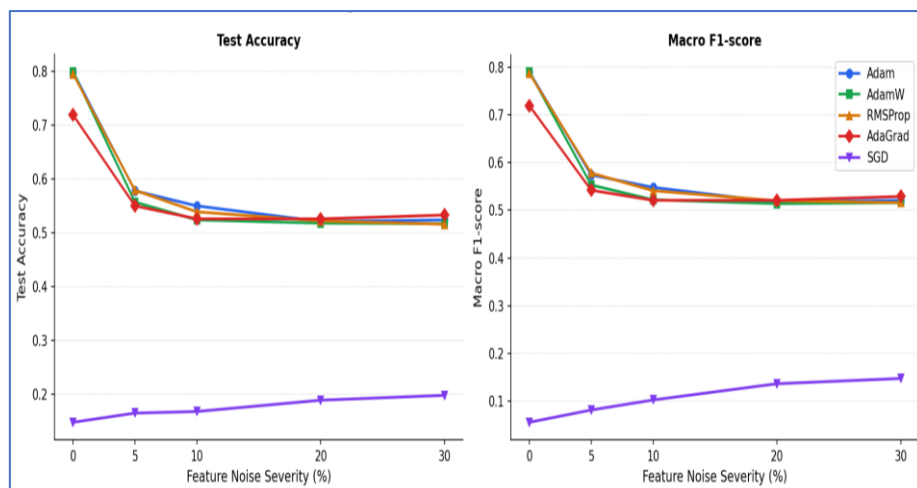


Figure 6 — Feature Noise Robustness: Accuracy & F1 vs Severity (0–30%)

Optimizer	5%	10%	20%	30%
Adam	0.578	0.550	0.521	0.524
AdamW	0.557	0.524	0.518	0.517
RMSProp	0.579	0.539	0.522	0.516
AdaGrad	0.550	0.526	0.526	0.533
SGD	0.165	0.168	0.189	0.198

Table 7a — Test Accuracy under Feature Noise by Severity Level

Optimizer	5%	10%	20%	30%
Adam	0.574	0.548	0.517	0.521
AdamW	0.553	0.522	0.514	0.516
RMSProp	0.578	0.541	0.520	0.516
AdaGrad	0.542	0.521	0.521	0.529
SGD	0.082	0.103	0.137	0.148

Table 7b — Macro F1-score under Feature Noise by Severity Level

The sharpest drop occurs between 5% and 10% severity: Adam falls from 0.578 to 0.550, a loss of 2.8 points in a single severity step, while losing only a further 2.9 points across the remaining 10%→30% range. This indicates that the GCN's sensitivity to feature corruption saturates quickly. The macro F1 values track test accuracy closely at every severity level, confirming the GCN does not develop a class-specific bias under feature noise. SGD's F1 score actually rises with severity (0.082 → 0.148), which is an artifact of its near-uniform random predictions becoming marginally less concentrated on majority classes as noise increases, not a sign of genuine improvement.

4.8 Edge Removal Robustness — Detailed Severity Analysis

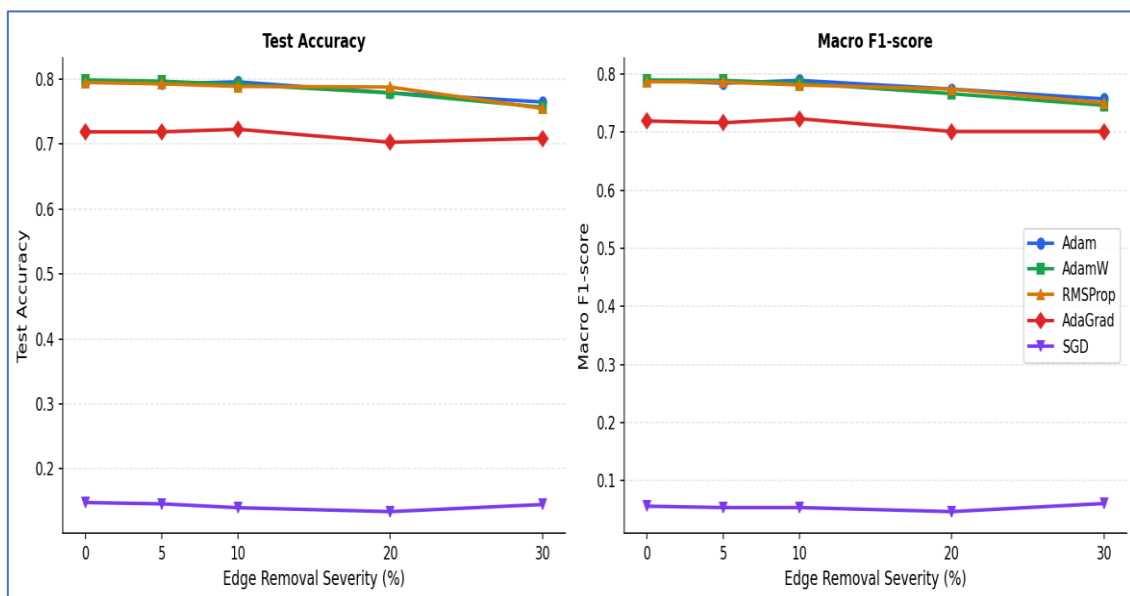


Figure 7 — Edge Removal Robustness: Accuracy & F1 vs Severity (0–30%)

Optimizer	5%	10%	20%	30%
Adam	0.793	0.796	0.779	0.765
AdamW	0.797	0.792	0.779	0.757
RMSProp	0.793	0.789	0.788	0.755
AdaGrad	0.719	0.723	0.703	0.709
SGD	0.146	0.140	0.134	0.145

Table 8a — Test Accuracy under Edge Removal by Severity Level

Optimizer	5%	10%	20%	30%
Adam	0.784	0.789	0.774	0.757
AdamW	0.789	0.784	0.766	0.746
RMSProp	0.786	0.781	0.774	0.750
AdaGrad	0.716	0.723	0.701	0.701
SGD	0.054	0.054	0.047	0.061

Table 8b — Macro F1-score under Edge Removal by Severity Level

Edge removal is the perturbation type that adaptive optimizers tolerate best across every severity level tested. Adam, AdamW, and RMSProp all remain within a narrow 0.755–0.797 accuracy band across the full 5%–30% range. AdamW briefly leads at the lowest severity (0.797 at 5%) before being overtaken by Adam from 10% onward. The macro F1 scores remain within 1-2 points of accuracy throughout, indicating balanced per-class performance is preserved even as edges are removed. SGD's F1 stays critically low (0.047–0.061) across all severities, confirming it never learns a usable decision boundary regardless of graph connectivity.

4.9 Fake Edge Addition Robustness — Detailed Severity Analysis

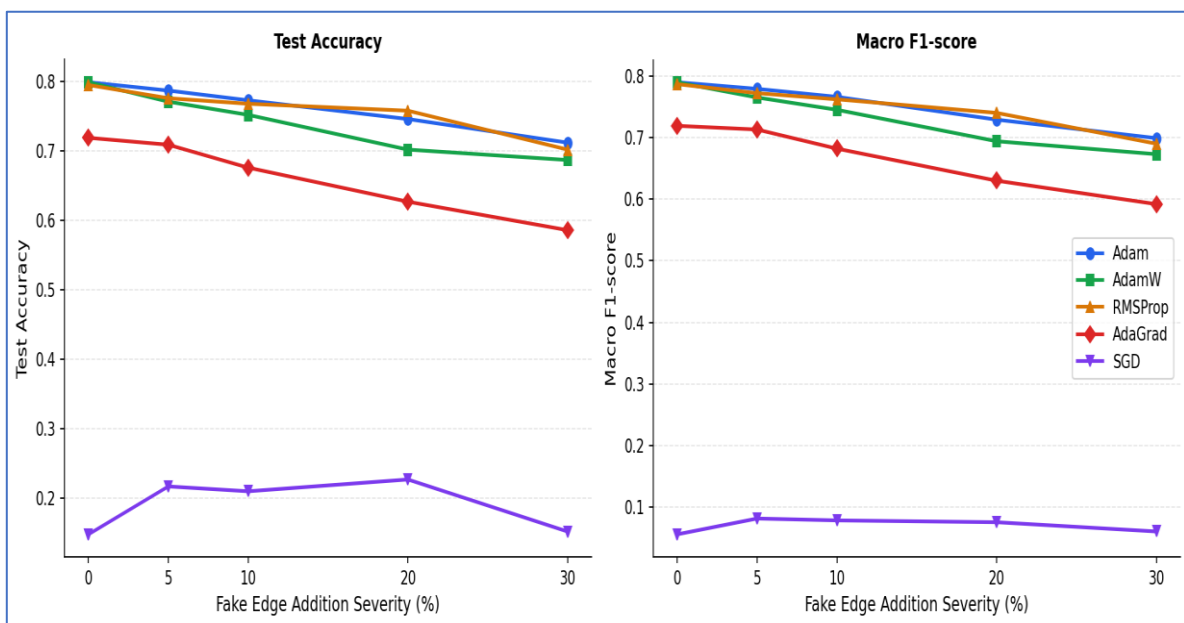


Figure 8 — Fake Edge Addition Robustness: Accuracy & F1 vs Severity (0–30%)

Optimizer	5%	10%	20%	30%
Adam	0.787	0.773	0.746	0.712
AdamW	0.771	0.752	0.702	0.687
RMSProp	0.776	0.768	0.758	0.702
AdaGrad	0.709	0.676	0.627	0.586
SGD	0.217	0.210	0.227	0.152

Table 9a — Test Accuracy under Fake Edge Addition by Severity Level

Optimizer	5%	10%	20%	30%
Adam	0.779	0.766	0.729	0.699
AdamW	0.765	0.745	0.694	0.673
RMSProp	0.772	0.762	0.740	0.690
AdaGrad	0.713	0.682	0.630	0.592
SGD	0.082	0.079	0.076	0.061

Table 9b — Macro F1-score under Fake Edge Addition by Severity Level

Fake edges produce a clean, monotonic degradation for every optimizer except SGD. Adam degrades from 0.787 to 0.712 (7.5-point drop in accuracy, 8.0-point drop in F1), the smallest relative loss among adaptive optimizers. AdaGrad shows the steepest decline of the four (0.709 $\rightarrow$ 0.586 accuracy, a 12.3-point drop), confirming that spurious connections disrupt its neighborhood aggregation more than either feature noise or edge removal. The close tracking between accuracy and F1 across all optimizers and severities confirms the GCN does not develop a systematic class bias under structural noise injection.

4.10 Training Time Analysis

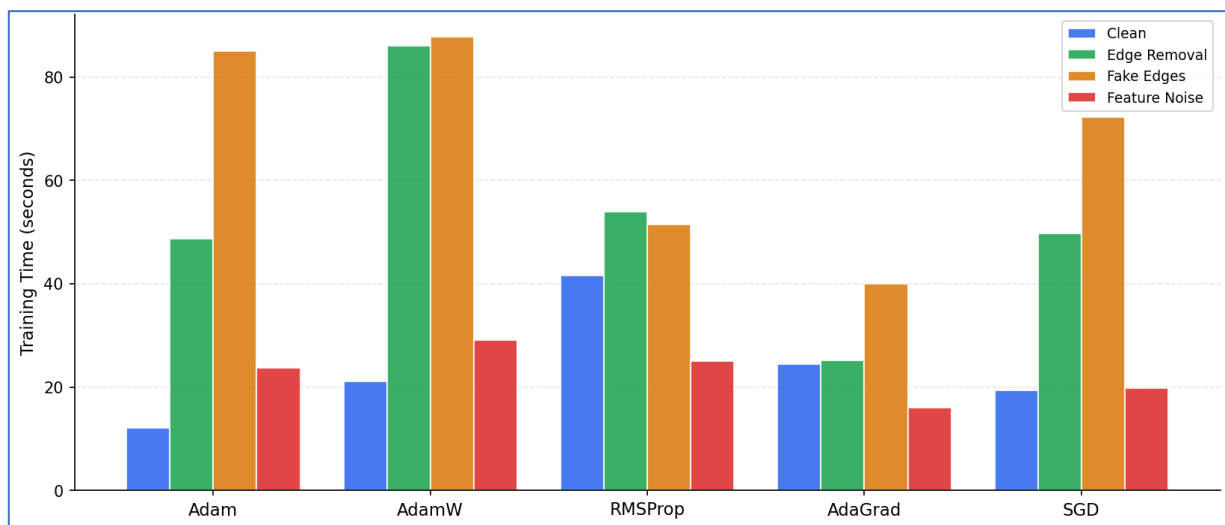


Figure 9 — Training Time per Optimizer Across Each Perturbation Condition

Optimizer	Clean (s)	Edge Rem. (s)	Fake Edges (s)	Feat. Noise (s)
Adam	12.1	48.6	84.9	23.7
AdamW	21.1	86.1	87.7	29.1
RMSProp	41.7	53.9	51.5	25.1
AdaGrad	24.5	25.1	39.9	16.1
SGD	19.4	49.7	72.2	19.8

Table 10 — Mean Training Time (seconds) per Optimizer and Condition

Training time varies considerably with perturbation type rather than remaining fixed per optimizer. Adam is fastest on the clean graph (12.1s) but its training time rises substantially under fake edge addition (84.9s), a 7-fold increase. AdamW shows the highest overall training time under structural perturbations (up to 87.7s). AdaGrad maintains the most consistent training time across all four conditions, ranging narrowly between 16.1s and 39.9s.

4.11 Accuracy Drop Across Perturbation Severities

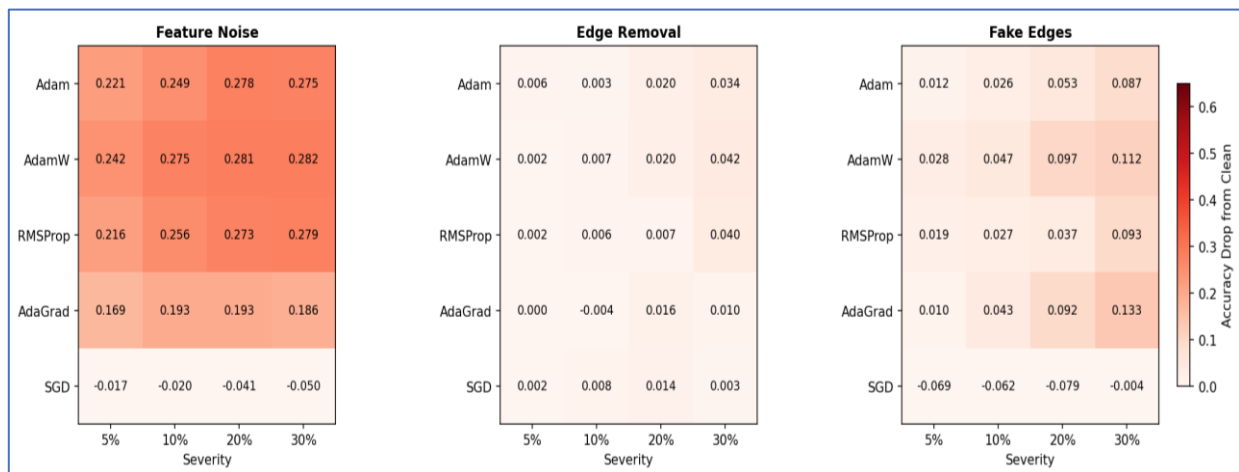


Figure10 — Accuracy Drop Heatmap by Perturbation Type and Severity Level

The heatmap confirms that feature noise produces the most uniform and severe degradation across all optimizers and severities, with drops exceeding 0.25 for every optimizer except SGD by 30% severity. Edge removal produces the smallest drop for Adam and RMSProp at every severity level, never exceeding 0.04. Fake edge addition shows an intermediate, steadily increasing degradation pattern.

4.12 Aggregate Performance Across All Conditions

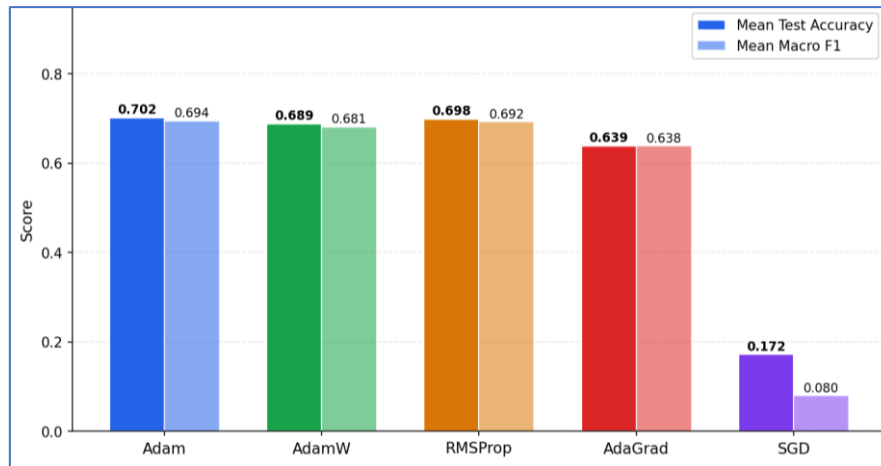


Figure 11 — Aggregate Mean Accuracy & F1 Across All Conditions

Optimizer	Clean	Edge Removal	Fake Edges	Feature Noise
Adam	0.799	0.783	0.755	0.543
RMSProp	0.795	0.781	0.751	0.539
AdamW	0.799	0.781	0.728	0.529
AdaGrad	0.719	0.714	0.650	0.534
SGD	0.148	0.141	0.202	0.180

Table 11 — Mean Test Accuracy per Optimizer and Perturbation Condition

Adam leads the aggregate ranking with the highest mean accuracy in three of the four conditions (clean, edge removal, fake edges) and the second-highest under feature noise. RMSProp is a consistent second across all conditions. AdamW performs strongly on clean and edge-removal conditions but falls behind under fake edge addition.

5. Discussion

5.1 Interpretation of Results

Adam as the consistent leader. Across all four experimental conditions, Adam ranks first or second in every comparison, and shows one of the smallest gradient norm spikes during training. Its combination of momentum and adaptive per-parameter scaling appears particularly well suited to the sparse, irregular gradient signals produced by GCN message passing on a small labeled set (140 nodes).

Adam vs AdamW: accuracy parity, different internal dynamics. Both achieve identical clean accuracy (0.799), but AdamW converges faster (epoch 157 vs Adam's 194) and reaches a much lower training loss (0.058 vs 0.316), at the cost of the widest train/validation gap (0.636) of any optimizer in this study. This pattern is consistent with AdamW fitting the small training set more tightly while gaining limited generalization benefit on a graph of Cora's size, and it shows a larger drop under fake edge addition (0.728 vs Adam's 0.755).

RMSProp's gradient behavior. RMSProp achieves the highest final validation accuracy (0.794) and the highest gradient L2 norm (0.300) among all optimizers — its per-parameter normalization keeps gradients from vanishing even after 190+ epochs, which may explain its continued late-stage learning. Its main drawback is training time on the clean graph (41.7s, more than 3x Adam's).

AdaGrad's incomplete convergence. AdaGrad reaches its plateau very late — at epoch 184 — still improving marginally until near the end of the 200-epoch budget. Combined with its small train/validation gap (0.244), this points to genuine underfitting rather than overfitting: AdaGrad's decaying effective learning rate slows it down before it can fully exploit the available training signal.

Feature noise as the universal challenge. All four competitive optimizers converge toward a similar degraded accuracy range (0.516–0.533) by 30% feature noise severity, regardless of their very different convergence speeds, gradient norms, and clean-graph accuracies. This indicates that this perturbation type challenges the two-layer GCN architecture itself rather than any specific optimizer's update rule.

SGD under the fixed protocol. SGD is formally classified as showing no useful convergence. Its near-zero train/validation gap and low final gradient norm (0.060) do not indicate stability but rather a model that never escapes the neighborhood of its random initialization, underlining how critical adaptive learning rates are for GCN training within a fixed-epoch, fixed-learning-rate budget.

5.2 Strengths and Weaknesses per Optimizer

Optimizer	Strengths	Weaknesses
Adam	Highest accuracy on every condition tested; best aggregate robustness; smallest gradient norm spikes; near-best convergence speed	Training time increases sharply under structural perturbations (up to 85s)

AdamW	Lowest training loss and final gradient norm among adaptive methods; second-fastest convergence (epoch 157)	Widest train/val gap (0.636), highest peak training time under structural noise; weaker under fake edges
RMSProp	Strong, consistent accuracy across all four conditions; second-best aggregate ranking; highest final validation accuracy on clean graph (0.794)	Slowest to truly plateau (epoch 193); highest clean-graph training time (41.7s)
AdaGrad	Smallest train/val gap (0.244), suggesting least overfitting; lowest final gradient norm among learning optimizers (0.108); competitive under edge removal	Reaches plateau very late (epoch 184); lower accuracy ceiling; steepest drop under fake edges
SGD	Lowest memory footprint tie; near-zero train/val gap (reflects failure to learn, not good generalization)	Accuracy remains near random-guess level (1/7 classes) across every condition; convergence status: no useful convergence

Table 12 — Comparative Strengths and Weaknesses

6. Limitations and Validity

- Single seed: all experiments use seed 42; multiple seeds would provide confidence intervals on every reported metric.
- Fixed learning rate: the protocol is fair for comparison purposes but may disadvantage SGD, which typically requires momentum or a learning rate schedule to perform competitively.
- One dataset: Cora is a standard benchmark but relatively small; PubMed and CiteSeer would test how these rankings generalize to larger and noisier citation graphs.
- One architecture: conclusions apply specifically to this 2-layer GCN and may not transfer directly to GAT or GraphSAGE architectures.
- Random, non-adversarial perturbations: the feature noise, edge removal, and fake edge conditions are randomly sampled rather than adversarially optimized attacks.

These limitations do not invalidate the project's conclusions. They define the scope of interpretation: this is a controlled optimizer comparison under graph noise for one architecture and one dataset, not a universal claim about every GNN or every citation network.

7. Conclusion

7.1 Synthesis

This study compared five optimization algorithms for training a 2-layer GCN on the Cora citation network under a fixed protocol and three perturbation regimes, covering 65 experimental runs plus a supplemental instrumented run tracking gradient norms and memory usage. Key findings:

1. Adam achieves the best or near-best accuracy in every one of the four tested conditions, combined with the lowest training time on the clean graph and the highest aggregate robustness (0.702 mean accuracy, 0.694 mean F1).
2. AdamW converges fastest, reaching within 1% of its best validation loss by epoch 157, and reaches the lowest training loss of any optimizer, but shows the widest generalization gap and loses ground specifically under fake edge addition.
3. RMSProp reaches the highest final validation accuracy (0.794) and the highest gradient L2 norm throughout training, at the cost of the highest clean-graph training time.
4. AdaGrad shows the smallest train/validation gap but reaches its plateau very late (epoch 184), reflecting genuine underfitting rather than overfitting; it remains the most competitive alternative under edge removal specifically.
5. SGD is formally classified as showing no useful convergence under this fixed protocol, remaining close to a random-guess baseline across all conditions tested.

7.2 Recommendations

Primary recommendation: Adam is the most reliable optimizer for GCN training on citation networks, offering the strongest accuracy across clean and perturbed conditions together with favorable training time on the unperturbed graph and stable gradient behavior.

When fast convergence matters: AdamW reaches within 1% of its best validation loss in the fewest epochs (157), making it attractive when the training budget itself is constrained — though this comes with a wider train/validation gap than Adam.

For graphs subject to structural noise: AdaGrad becomes a more attractive option specifically under edge-removal scenarios, where its accuracy gap with Adam narrows substantially; Adam and AdamW remain close competitors at low edge-removal severity, with Adam pulling ahead only from 10% severity onward.

Future work could extend this comparison to GAT and GraphSAGE architectures, explore adaptive learning rate schedulers, evaluate multiple random seeds for statistical confidence, and study how these optimizer rankings evolve on larger citation graphs such as CiteSeer and PubMed.

8. Bibliography

- [1] Kipf, T. N., & Welling, M. (2017). Semi-Supervised Classification with Graph Convolutional Networks. ICLR 2017. arxiv.org/abs/1609.02907
- [2] Hamilton, W., Ying, Z., & Leskovec, J. (2017). Inductive Representation Learning on Large Graphs. NeurIPS 2017. arxiv.org/abs/1706.02216
- [3] Velickovic, P., et al. (2018). Graph Attention Networks. ICLR 2018. arxiv.org/abs/1710.10903
- [4] Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. ICLR 2015. arxiv.org/abs/1412.6980
- [5] Loshchilov, I., & Hutter, F. (2019). Decoupled Weight Decay Regularization. ICLR 2019. arxiv.org/abs/1711.05101
- [6] Hinton, G., Srivastava, N., & Swersky, K. (2012). RMSProp. Neural Networks for Machine Learning, Lecture 6e, Coursera.
- [7] Zugner, D., Akbarnejad, A., & Gunnemann, S. (2018). Adversarial Attacks on Neural Networks for Graph Data. KDD 2018. arxiv.org/abs/1805.07984
- [8] Fey, M., & Lenssen, J. E. (2019). Fast Graph Representation Learning with PyTorch Geometric. ICLR Workshop 2019. arxiv.org/abs/1903.02428

9. Annexes

9.1 Code Repository

GitHub: <https://github.com/Maaskk/gnn-optimization-under-graph-perturbations-cora>

- `src/gnn_robustness/` — GCN model, dataset loading, perturbation functions, training loop, metrics
- `scripts/` — Reproducible experiment scripts and frontend data generation
- `results/` — All CSV result tables, `report_tables/`, and generated figures
- `docs/` — Interactive static frontend (HTML/JS)
- `tests/` — Regression tests for metrics, perturbations, and result schema

9.2 Interactive Dashboard

Live site: <https://maaskk.github.io/gnn-optimization-under-graph-perturbations-cora/>

The dashboard features an animated Cora graph visualization, CSV-driven optimizer ranking, a perturbation chamber with severity sliders, a loss-drop heatmap, and convergence analysis panels, including a dedicated protocol page documenting model, optimization, and evaluation settings.

9.3 Result Files

- `all_results.csv` — Full experiment table (65 rows: 5 optimizers x 13 conditions)
- `clean_loss_history.csv` — Per-epoch train/validation loss and accuracy for all optimizers on the clean graph
- `clean_optimizer_results.csv` — Final clean-graph results per optimizer
- `edge_removal_results.csv` / `fake_edge_addition_results.csv` / `feature_noise_results.csv` — Per-condition results at 5%, 10%, 20%, 30% severity
- `final_optimizer_summary.csv` — Aggregate ranking table by optimizer and condition
- `cora_dataset_summary.csv` — Dataset statistics
- `gradient_norm_history.csv` — Per-epoch gradient L2 norm for all optimizers (clean graph)
- `resource_usage.csv` — Peak memory usage and final gradient norm per optimizer
- `report_tables/convergence_epochs.csv` — Convergence speed and status per optimizer
- `report_tables/train_validation_gap.csv` — Generalization gap metrics per optimizer
- `report_tables/detailed_by_severity.csv` — Full accuracy/F1/loss/time breakdown by severity
- `report_tables/macro_f1_*_by_severity.csv` — Macro F1-score by severity for each perturbation type

9.4 Jupyter Notebook

A complete Jupyter notebook is available in `notebooks/` demonstrating the GCN model definition, training loop, perturbation functions, gradient norm and memory instrumentation, every figure included in this report, and the full comparison pipeline, runnable on Google Colab.